

Shippin

Project Documentation

JavaFX shipment cost estimation and shipment management application

Skupina 6

Marko Bukovina, Juraj Černička, Martin Kvietok, Filip Brutovský, Maximilián Bumbera, Marcel Novomeský,

Ondrej Krajčovič, Viktória Latičová, Martin Okša, Richard Zahoranský, Iveta Lešková

Item	Description
GitHub repository	github.com/mbukovina127/vava-project
Jira	https://vava-team-six.atlassian.net/jira/software/projects/VAVA
Application type	Desktop GUI application built with Java, Gradle and JavaFX
Database	PostgreSQL accessed through JDBC
Main domain	Shipment price estimation, shipment tracking, warehouse management and price-list management
Document purpose	Technical documentation and requirement-alignment documentation for the Shippin application

Prepared from the project repository, implementation structure and supplied design diagrams. The document is written as an editable technical description for further project work.

Contents

1. Project overview
2. Requirement alignment matrix
3. User roles, use cases and screens
4. Layered architecture and package separation
5. Business logic: shipment cost estimation
6. Shipment lifecycle and state management
7. Persistence, JDBC and database schema
8. Import/export, XML and CSV infrastructure
9. Validation, regular expressions, filtering and security
10. Localization and logging
11. Collections, data structures and Lombok
12. Design patterns used in the project
13. Build, dependencies and testing support
14. Diagram catalogue and appendix

1. Project overview

Shippin is a desktop application for calculating shipment costs and managing shipments in a logistics-oriented environment. The application is intended for company users who need to estimate delivery prices from practical shipment parameters such as postal code, warehouse, weight, volume, fuel surcharge, toll and additional services.

The system covers the complete workflow from account access to shipment calculation, cost breakdown, shipment storage, route/map visualization, warehouse management and administrative user/service management. It is implemented as a JavaFX GUI application, with Gradle build configuration and PostgreSQL persistence through JDBC.

Main functional areas

- User registration and login.
- Role-based access for User, Power user and Administrator.
- Shipment cost estimation and cost-breakdown review.
- Shipment creation, detail view, state changes and history.
- Warehouse management, including price lists and region tables.
- Small-package price-list handling.
- Service management for configurable cost modifiers and default costs.
- Import/export support for CSV and XML data files.
- Map-related shipment route and location functionality.

Core implementation packages

Package/resource	Role in the application
src/main/resources/views	FXML screens that define the JavaFX presentation layer.
org.shippin.controller	JavaFX controllers that coordinate user actions and view state.
org.shippin.services	Application services and business logic.
org.shippin.database and org.shippin.database.dao	JDBC connection and DAO classes for database access.
org.shippin.domain	Encapsulated domain entities such as Shipment, Warehouse, User, PriceList and AdditionalService.
org.shippin.dto	Enums and data-transfer/support types such as Screens and RegPattern.
org.shippin.infrastructure.csv and org.shippin.infrastructure.xml	Import/export parsers for price lists, region tables and small-price lists.
org.shippin.infrastructure.validation	Validation logic for imported tables.
org.shippin.util and org.shippin.util.io	Utility classes for ranges, conversion, file selection and file I/O.

2. Requirement alignment matrix

This section maps the required technical topics to the concrete places where they appear in the Shippin project. It is intended to make the relationship between the application and the assignment requirements explicit.

Requirement	Project evidence
Multi-layer architecture: presentation, business and data/persistence layers	Separated packages: org.shippin.controller and FXML views for presentation; org.shippin.services for business/application logic; org.shippin.database and org.shippin.database.dao for JDBC persistence.
Presentation layer separated from database access	The architecture is defined around a Controller -> Service -> DAO -> DB flow. DAO classes are isolated in the persistence package and database access is encapsulated behind the business/data layers.
Collections and suitable data structures	List/ArrayList for shipments, users, services and price-list entries; Map/HashMap for region lookup and summaries; LinkedHashMap/LinkedHashSet where stable UI ordering and selected-service state are useful; Comparator and streams for filtering and sorting.
Logging of business logic and exceptions/errors	@Log4j2 is used across services, controllers and DAO classes. Important operations such as navigation, shipment creation, saving, state changes, user changes and error handling are logged with info/warn/error-level messages.
Localization: English and Slovak	Resource bundles messages_en.properties and messages_sk.properties are stored in src/main/resources/i18n. NavigationService loads ResourceBundle and provides it to FXMLLoader.
XML import/export	XmlParser interface and concrete XML parsers in org.shippin.infrastructure.xml. WarehouseParsingService supports XML read/write for price lists, region tables and small price lists.
Regular expressions and filtering	RegPattern enum stores compiled regex patterns; InputValidator and ErrorHandler use them for email, name, password, numbers, percentage and postal-code validation. Search/filter fields are used in user and shipment screens.
JDBC database access without ORM	build.gradle.kts includes the PostgreSQL JDBC driver. DBConnector uses DriverManager and java.sql.Connection. DAO classes use PreparedStatement and ResultSet. No JPA, Hibernate, EclipseLink, MyBatis, Spring or Spring Boot dependency is used.
Input validation, security and encapsulated data models	Validation is implemented before business actions are executed. PreparedStatement is used for parameterized SQL. Domain models use private fields and getter/setter access, with Lombok reducing boilerplate.
Desktop GUI application	JavaFX is configured in Gradle and views are FXML files. The application is not Android and not Spring/Spring Boot.
Three user types/actors	Role enum defines USER, POWER_USER and ADMIN. Screens enum maps screen access to required role. Guest appears in the use-case model for registration and login.
5-8 or more application screens	The application includes more than the minimum: Login, Register, Menu, CostEstimation, CostBreakdown, MyShipments, ShipmentDetail, MapOfShipments, WarehouseManagement, EditWarehouse, SmallPriceListView, UserManagement, ServicesManagement and Daily summaries.
Lombok usage	Lombok dependency is configured in Gradle. The code uses annotations such as @Getter, @Setter, @Data, @AllArgsConstructor, @NoArgsConstructor and @Log4j2.

3. User roles, use cases and screens

The application is designed around a role hierarchy. A basic User can estimate shipment costs and work with shipments. A Power user extends the user role with shipment and warehouse-related operational tasks. An Administrator extends the previous roles with user and service administration. A Guest can register or log in.

Role / actor	Purpose	Main capabilities
Guest	Unauthenticated visitor	Register and log in.
User	Regular authenticated application user	Estimate shipment cost, view cost breakdown, create shipment and view own shipments.
Power user	Operational employee / advanced user	Manage shipments, view shipment routes, work with warehouse-related operational screens.
Administrator	Management and administration role	Manage users, warehouses, services, price-list data and administrative overview screens.

Screen inventory

Screen/resource	Role in the project
Login.fxml	User authentication entry point.
Register.fxml	User account creation.
Menu.fxml	Main navigation screen after login.
CostEstimation.fxml	Shipment input form and cost-estimation workflow.
CostBreakdown.fxml	Detailed view of calculated cost components and save action.
MyShipments.fxml	List, filter, sort and open user shipments.
ShipmentDetail.fxml	Shipment details, state changes, route/map data and history.
MapOfShipments.fxml	Map view for active shipments.
WarehouseManagement.fxml	Warehouse list, import/export, replacement and deletion actions.
EditWarehouse.fxml	Warehouse details and editable warehouse table data.
SmallPriceListView.fxml	Small-package price list viewing/management.
UserManagement.fxml	Administrator user management and role assignment.
ServicesManagement.fxml	Administrator additional-service management.
DailyCostsSummaries.fxml / DailyCostsSummaryDetail.fxml	Cost summaries and daily shipment overview.

The use-case model in the appendix summarizes the role relationships and the main use cases. The Screens enum in `org.shippin.dto` represents the role-based screen access model in the implementation.

4. Layered architecture and package separation

Shippin uses a layered architecture with clear separation between GUI, business logic and persistence. This organization is visible both in the package structure and in the diagrams. The main rule is that UI code is responsible for interaction and navigation, services coordinate business operations, and DAOs own SQL/database access.

Layer	Packages/classes	Responsibilities
Presentation layer (GUI)	src/main/resources/views, org.shippin.controller	FXML layout, event handlers, screen interaction, displaying validation messages, navigation requests and form-state management.
Business layer (application logic)	org.shippin.services	Shipment calculation, shipment save/update workflows, user session and role logic, warehouse operations, parsing/export orchestration and navigation policy.
Data/persistence layer	org.shippin.database, org.shippin.database.dao	PostgreSQL connection, SQL queries, transactions, object mapping and DAO methods.
Domain and support layer	org.shippin.domain, org.shippin.dto, org.shippin.util, org.shippin.infrastructure	Domain entities, enums, parsers, validators, file helpers and formatted table objects.

Layered request flow

1. The user interacts with a JavaFX screen defined in FXML.
2. The matching controller reads input, invokes validation and calls a service operation.
3. The service applies business rules and coordinates persistence operations.
4. The DAO executes parameterized SQL through JDBC and maps database rows to domain objects.
5. The service returns the domain result to the controller, which updates the UI or navigates to the next screen.

Relevant implementation locations

```
src/main/resources/views/*.fxml
src/main/java/org/shippin/controller/*Controller.java
src/main/java/org/shippin/services/*Service.java
src/main/java/org/shippin/database/DBConnector.java
src/main/java/org/shippin/database/dao/*DAO.java
```

The Component diagram and Layered viewpoint in the appendix illustrate the same separation from different levels of abstraction.

5. Business logic: shipment cost estimation

The central business feature is shipment cost estimation. The user enters a destination postal code, weight, volume, fuel surcharge coefficient, toll coefficient, source warehouse and selected additional services. The application calculates a base price and then applies coefficient and service-based cost components.

Main implementation class

`src/main/java/org/shippin/services/ShipmentService.java`

Cost-estimation steps

1. Create a Shipment object containing destination postal code, warehouse, weight, volume, fuel surcharge, toll and selected services.
2. If the shipment weight is above 30 kg, find the destination region by matching the postal code against the warehouse region table.
3. For standard shipments, search the price list by region and choose the appropriate price threshold based on weight and volume.
4. For small packages, use the small-package price list and choose the matching weight threshold.
5. Calculate the base cost, apply the fuel and toll coefficients, apply selected service modifiers and add default service costs.
6. Return the Shipment object with totalCost populated and state initialized.

Business methods used in the calculation

Method / responsibility	Meaning
<code>createShipment(...)</code>	Builds the shipment entity, loads warehouse/service data, calculates base and total costs.
<code>calculateBaseCost(Shipment)</code>	Chooses either regular warehouse price list or small-package price list.
<code>calculateTotalCost(Shipment, baseCost)</code>	Applies fuel/toll coefficients and service cost modifiers/default costs.
<code>calculateFuelCost / calculateTollCost / calculateServiceCost</code>	Helper calculations used by the cost-breakdown screen.
<code>findRegionForPostalCode(...)</code>	Matches destination postal code to the correct region-table entry.
<code>findCostInPriceList(...)</code> / <code>findCostInSmallPriceList(...)</code>	Finds the best matching price threshold.

The Activity diagram - Cost estimation and Class diagram - ShipmentService in the appendix show the same logic visually.

6. Shipment lifecycle and state management

Shipments follow a controlled lifecycle. The State enum defines the possible states and the allowed transitions. This keeps the workflow predictable and prevents arbitrary state changes.

```
src/main/java/org/shippin/domain/enums/State.java
```

```
src/main/java/org/shippin/services/ShipmentService.java
```

State	Meaning	Allowed next states
NOT_READY	Shipment is created but not ready for delivery.	READY_FOR_DELIVERY, CANCELED
READY_FOR_DELIVERY	Package is prepared for delivery.	BEING_DELIVERED, CANCELED
BEING_DELIVERED	Shipment is currently in delivery.	DELIVERED, FAILED
DELIVERED	Shipment was delivered successfully.	Terminal state
CANCELED	Shipment was canceled before delivery.	Terminal state
FAILED	Shipment delivery failed during the journey.	Terminal state

State changes are also persisted into shipment history. `ShipmentService.updateShipmentState(...)` changes the current shipment status, inserts a `ShipmentHistory` entry and uses a transaction so the status and history entry are stored together.

The State diagram in the appendix gives a visual view of the same lifecycle.

7. Persistence, JDBC and database schema

The application uses PostgreSQL through JDBC. The persistence layer is located under `org.shippin.database` and `org.shippin.database.dao`. Database connection setup is encapsulated in `DBConnector`, while individual DAO classes contain SQL operations for specific domain areas.

```
src/main/java/org/shippin/database/DBConnector.java
src/main/java/org/shippin/database/Config.java
src/main/java/org/shippin/database/dao/BaseDAO.java
src/main/java/org/shippin/database/dao/UserDAO.java
src/main/java/org/shippin/database/dao/ShipmentDAO.java
src/main/java/org/shippin/database/dao/WarehouseDAO.java
src/main/java/org/shippin/database/dao/PriceListDAO.java
src/main/java/org/shippin/database/dao/RegionDAO.java
src/main/java/org/shippin/database/dao/AdditionalServiceDAO.java
```

JDBC responsibilities

Class	Persistence responsibility
DBConnector	Creates the PostgreSQL JDBC connection using DriverManager and database configuration properties.
Config	Loads database connection properties from the config resource.
BaseDAO	Provides the shared JDBC Connection and transaction helper methods such as commit and rollback.
UserDAO	User insert, authentication, lookup, role update, list and delete operations.
ShipmentDAO	Shipment read/write, history, services attached to shipments and status updates.
WarehouseDAO	Warehouse lookup, insert/update/delete and full warehouse operations with related tables.
PriceListDAO	Regular and small-package price-list storage and retrieval.
RegionDAO	Region-table and postal-code range storage and retrieval.
AdditionalServiceDAO	CRUD operations for configurable services/products/payments.

Database model overview

- users stores application users, role and active status.
- warehouse stores warehouse metadata, location and price-list reference.
- shipment stores shipment cost parameters, total cost, status and links to user/warehouse.
- history stores timestamped shipment state changes.
- service and service_list store additional services and the shipment-service relation.
- region, postal_code and postal_code_list represent warehouse region tables and postal-code ranges.
- parameter_list stores regular shipment price-list entries.
- sp_price_list stores small-package prices.

The database schema diagram in the appendix shows the table relationships.

8. Import/export, XML and CSV infrastructure

The application supports structured import and export of warehouse-related data. This includes price lists, region tables and small-package price lists. CSV and XML are handled through parser interfaces and concrete parser implementations.

```
src/main/java/org/shippin/services/WarehouseParsingService.java
src/main/java/org/shippin/infrastructure/csv/CsvParser.java
src/main/java/org/shippin/infrastructure/csv/PriceListCsvParser.java
src/main/java/org/shippin/infrastructure/csv/RegionTableCsvParser.java
src/main/java/org/shippin/infrastructure/csv/SmallPriceListCsvParser.java
src/main/java/org/shippin/infrastructure/xml/XmlParser.java
src/main/java/org/shippin/infrastructure/xml/PriceListXmlParser.java
src/main/java/org/shippin/infrastructure/xml/RegionTableXmlParser.java
src/main/java/org/shippin/infrastructure/xml/SmallPriceListXmlParser.java
src/main/java/org/shippin/util/io/FilePicker.java
src/main/java/org/shippin/util/io/TextFileHandler.java
```

Data type	CSV support	XML support	Formatted model
Price list	PriceListCsvParser	PriceListXmlParser	PriceListFormatted / PriceListRow
Region table	RegionTableCsvParser	RegionTableXmlParser	RegionTableFormatted / RegionTableRow
Small price list	SmallPriceListCsvParser	SmallPriceListXmlParser	SmallPriceListFormatted / SmallPriceListRow

WarehouseParsingService acts as the import/export coordinator. It reads file content through TextFileHandler, chooses the correct parser according to the file extension and writes exported table content back to the selected target file. This gives the project a clean extension point for multiple file formats.

The Class diagram - CSV and Activity diagram - warehouse export in the appendix document this part visually.

9. Validation, regular expressions, filtering and security

Input validation appears in both GUI workflows and import workflows. GUI validation prevents invalid form values from being passed into business operations. Import validation checks that CSV/XML tables have the expected structure before they are converted into domain objects or stored in the database.

```
src/main/java/org/shippin/dto/RegPattern.java
src/main/java/org/shippin/controller/utils/InputValidator.java
src/main/java/org/shippin/controller/utils/ErrorHandler.java
src/main/java/org/shippin/infrastructure/validation/*Validator.java
```

Validation area	Implementation
Email	RegPattern.EMAIL and ErrorHandler.validateEmail(...)
Password	InputValidator checks minimum length, uppercase, lowercase, digit and special character.
Name and surname	RegPattern.NAME allows letters and selected separators.
Postal code	RegPattern.POSTAL_CODE supports Slovak postal-code style with optional space.
Numbers and percentages	RegPattern.POSITIVE_DOUBLE and RegPattern.PERCENT plus numeric range checks.
Imported price-list data	PriceListValidator validates table rows and zone columns.
Imported region-table data	RegionTableValidator validates zones and postal-code ranges.
Imported small price list	SmallPriceListValidator validates expected rows and prices.

Filtering and search

- UserManagementController provides search by user name and filters the displayed user list.
- MyShipmentsController provides search by shipment ID and sorting by time, cost and state.
- Filtering uses Java collections, stream operations and Comparator logic.

Security and SQL-injection protection

- DAO methods use PreparedStatement and bound parameters for values received from users or application state.
- Validation is applied before database operations in form-based workflows.
- Domain objects expose private fields through getters/setters instead of public mutable fields.
- User access is controlled by Role and Screens access checks.

10. Localization and logging

Localization is implemented with Java ResourceBundle files. This allows the same FXML screens and controllers to display English or Slovak text depending on the selected locale. Labels, button texts, error messages, validation messages and service-management texts are stored in properties files.

```
src/main/resources/i18n/messages_en.properties
src/main/resources/i18n/messages_sk.properties
src/main/java/org/shippin/services/NavigationService.java
```

Localization flow

1. NavigationService keeps the active ResourceBundle.
2. setLocale(Locale) switches the bundle.
3. FXMLLoader receives the active bundle through loader.setResources(bundle).
4. Controllers and helper classes read translated strings using NavigationService.getBundle().
5. FXML files can use resource keys to display localized labels and buttons.

Logging is used to record business events and failures in a developer-readable way. Log messages are present in navigation, user management, shipment calculation, shipment persistence and warehouse operations. The project uses Log4j2 dependency and Lombok @Log4j2 to reduce repeated logger boilerplate.

Area	Example logging responsibility
NavigationService	Logs navigation and access-denied situations.
ShipmentService	Logs shipment creation, saving and state changes.
WarehouseDAO / WarehouseService	Logs warehouse insert, update, delete and transaction rollback paths.
UserManagementController / UserService	Logs user loading, role changes, deletion and registration/authentication events.
Controllers	Log GUI workflow exceptions and user-action failures.

11. Collections, data structures and Lombok

The project uses standard Java collections according to the type of data being stored and processed. Lists represent ordered records, maps represent lookup/grouping operations, sets represent unique selections and streams/comparators support filtering and sorting.

Data structure	Where it is used	Reason
List / ArrayList	Users, shipments, services, price-list entries, region-table entries, table rows.	Natural choice for ordered collections of domain objects and rows.
Map / HashMap	RegionDAO groups region table entries by region ID; ShipmentService creates daily summaries by LocalDate.	Efficient grouping and key-based lookup.
LinkedHashMap	CostEstimationController maps UI controls to AdditionalService while preserving display order.	Stable iteration order for dynamic GUI controls.
LinkedHashSet / Set	Cost-estimation form state stores checked service IDs without duplicates.	Unique selected-service identifiers.
Streams and Comparator	MyShipmentsController filters and sorts shipment rows.	Readable filtering/sorting over collections.

Lombok is used to reduce repetitive model and logger code. The Gradle build configures Lombok as compileOnly and annotationProcessor. Domain and support classes use annotations such as @Getter, @Setter, @Data, @AllArgsConstructor and @NoArgsConstructor. Classes using logging can use @Log4j2 to generate the logger field automatically.

```
build.gradle.kts - org.projectlombok:lombok dependency and annotation processor
src/main/java/org/shippin/domain/Shipment.java - @Getter, @Setter, @AllArgsConstructor, @NoArgsConstructor
src/main/java/org/shippin/domain/CoreShipmentInfo.java - @Data, @AllArgsConstructor, @NoArgsConstructor
src/main/java/org/shippin/dto/RegPattern.java - @Getter
src/main/java/org/shippin/services/NavigationService.java - @Log4j2 and @Setter
```

12. Design patterns used in the project

The project uses several common application-design patterns. Some are explicit class-level patterns, while others appear as architectural patterns across packages.

Pattern	Where used	Purpose in Shippin
Layered Architecture	controller / services / database.dao packages	Separates GUI, business logic and persistence. Improves maintainability and makes the application easier to test and extend.
MVC / JavaFX controller pattern	FXML files + matching controller classes	FXML defines views, controllers handle user events, services/domain models contain business data and logic.
Service Layer	ShipmentService, UserService, WarehouseService, AdditionalServicesService, WarehouseParsingService	Provides business operations as a clean API for GUI workflows.
DAO pattern	UserDAO, ShipmentDAO, WarehouseDAO, PriceListDAO, RegionDAO, AdditionalServiceDAO	Encapsulates SQL, ResultSet mapping and persistence operations behind domain-focused methods.
Singleton	DBConnector.getInstance(), DAO getInstance() methods, service getInstance() methods	Provides a single shared access point for connection/DAO/service instances where appropriate.
Data Mapper	ShipmentDAO mapShipment/mapBriefShipment, AdditionalServiceDAO readService	Converts database rows into domain objects and keeps SQL mapping details out of business logic.
State Machine	State enum and allowedTransitions()	Defines valid shipment status transitions and terminal states.
Strategy / Parser abstraction	CsvParser and XmlParser interfaces with concrete price-list/region-table/small-price-list parsers	Allows the same import/export workflow to support different file formats.
DTO / View model	BriefWarehouse, BriefShipment, formatted table classes and row classes	Carries simplified or presentation-focused data between layers and views.
Facade / Coordinator	WarehouseParsingService, ShipmentService	Coordinates multiple lower-level operations through one business-facing method.
Observer / event-listener style	JavaFX properties and listeners in controllers	Reacts to UI state changes such as search text, sorting choice or focus changes.

The diagrams in the appendix illustrate these patterns at different abstraction levels: use cases show actor behavior, layered views show application structure, class diagrams show domain and parsing abstractions, and activity/state diagrams show process logic.

13. Build, dependencies and testing support

The build is configured with Gradle. The application entry point is defined as `org.shippin.app.Launcher`. JavaFX modules are configured through the OpenJFX Gradle plugin, and dependencies cover JDBC, logging, JSON support, map visualization, PDF support, testing and Lombok. Also, we used continuous integration for automatized testing in GitHub.

Dependency / plugin area	Use in the project
java and application plugins	Build and run the desktop application.
org.openjfx.javafxplugin	Provides JavaFX module configuration for controls, FXML, web and swing.
PostgreSQL JDBC driver	Connects the Java application to PostgreSQL through JDBC.
Log4j2	Application and business logging.
Lombok	Generates getters, setters, constructors, data methods and loggers.
Gson	JSON-related support where data serialization/deserialization is needed.
JXMapView	Map visualization support.
PDFBox	PDF-related output support.
JUnit, jqwik, TestFX/Monocle, JaCoCo	Unit/property/UI testing support and code coverage reporting.

Build and test commands

```
./gradlew run
./gradlew test
./gradlew jacocoTestReport
```

Technology stack summary

- Language/platform: Java with Gradle.
- GUI: JavaFX with FXML views.
- Database: PostgreSQL through JDBC.
- Architecture: layered desktop application with service and DAO layers.
- Data exchange: CSV and XML parsers.
- Localization: Java ResourceBundle properties files.
- Logging: Log4j2.
- Code generation: Lombok.

14. Diagram catalogue and appendix

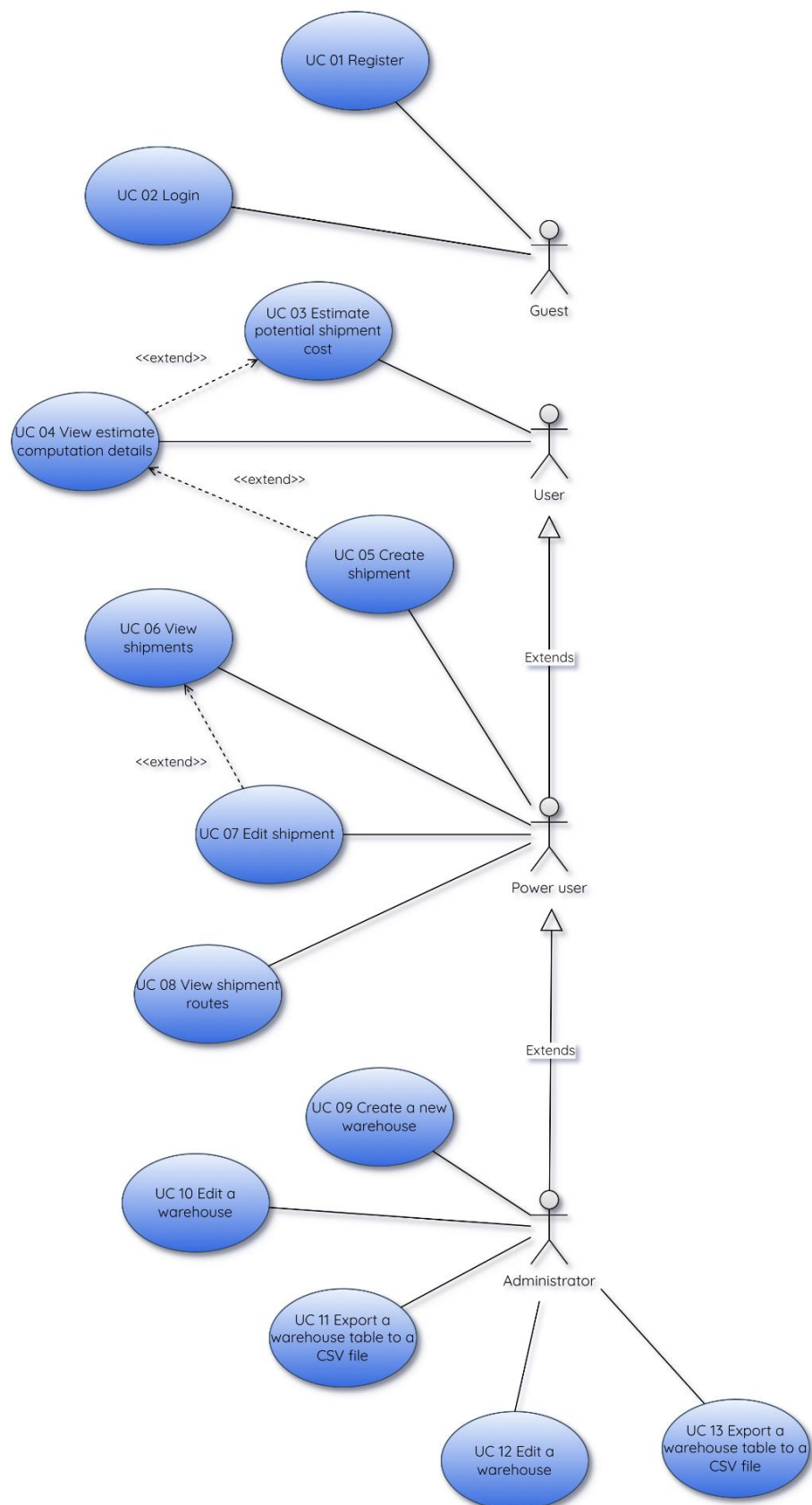
The following diagrams are included as full-page figures in the appendix. They are placed after the main text so they can be edited, moved or replaced independently in Microsoft Word.

No.	Diagram	Purpose
Figure 1	Use case model	Shows the main actors and use cases: registration, login, cost estimation, cost breakdown, shipment creation, shipment viewing/editing, route viewing, warehouse management and export operations.
Figure 2	Application usage viewpoint	Connects business actors and roles with application services such as services management, warehouse management, shipment management and shipment estimation.
Figure 3	Layered viewpoint	Shows how business roles, business services, application services, components and technology elements relate to each other.
Figure 4	Technology layer viewpoint	Shows the deployment idea: user device with Java runtime and JAR file communicating over TCP/IP with a server VM running PostgreSQL.
Figure 5	Component diagram - application services	Shows the main service components, API/controller layer, external map provider and database layer.
Figure 6	Activity diagram - shipment cost estimation	Shows the business steps of the shipment cost calculation process.
Figure 7	Activity diagram - warehouse export	Shows the export process from query result through warehouse conversion and CSV creation.
Figure 8	State diagram - shipment lifecycle	Shows permitted shipment states and transitions from creation to delivered/cancelled/failed outcomes.
Figure 9	Class diagram – ShipmentService cost estimation	Shows the central role of ShipmentService.createShipment and the relation to Shipment and estimateCost behavior.
Figure 10	Class diagram – warehouse and user domain objects	Shows the warehouse and user domain objects, and the crucial objects necessary for their implementation.
Figure 11	Class diagram – shipment and shipment daily summary domain objects, and DAO objects.	Shows DAO objects, shipment domain object and a ShipmentsDailySummary domain object.
Figure 12	Class diagram - CSV/table infrastructure	Shows formatted table classes, row classes, file utilities and CSV parser abstractions.
Figure 13	Database schema	Shows PostgreSQL tables for users, warehouses, shipments, history, services, price lists, regions and postal-code ranges.

Appendix A - Diagrams

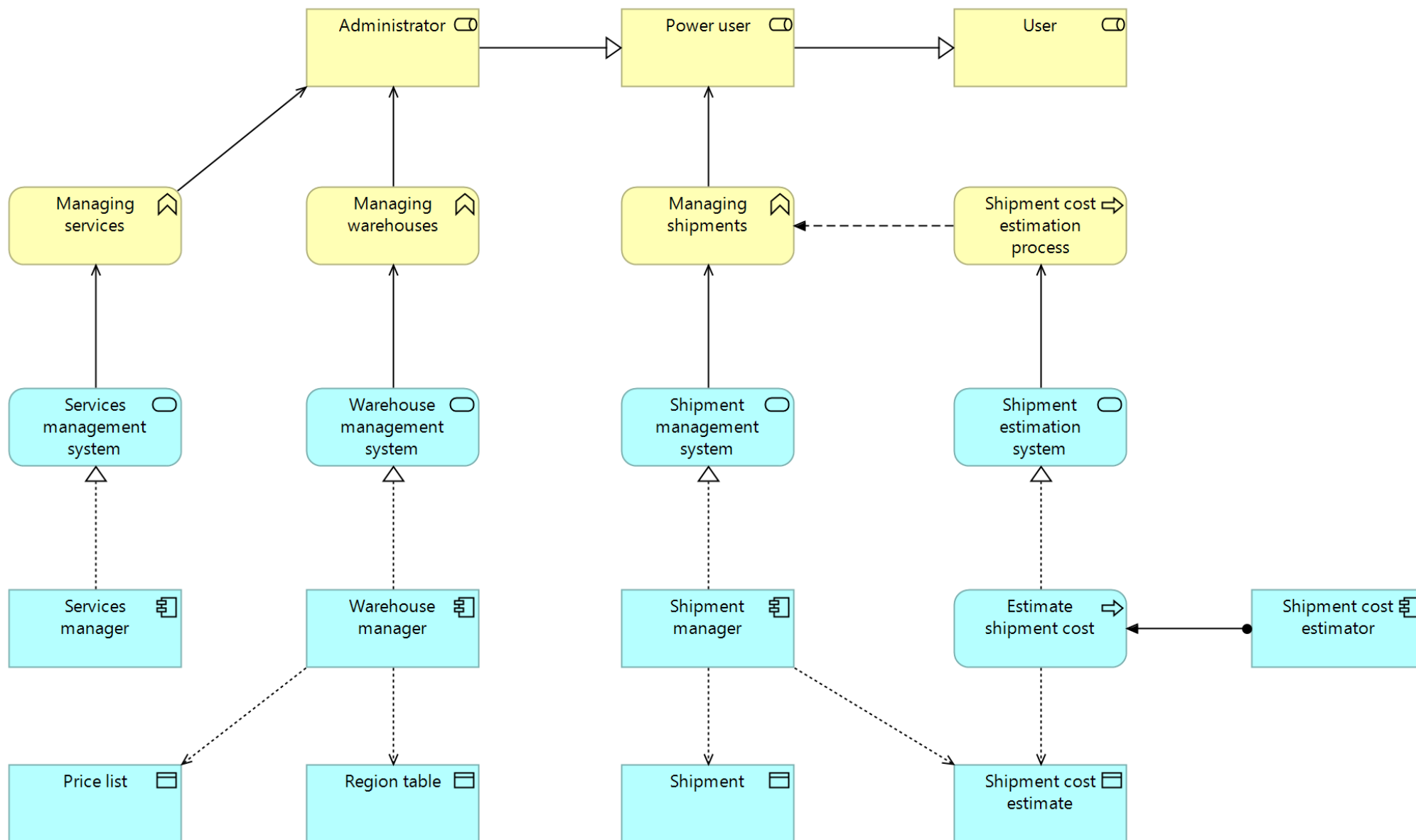
This appendix contains all supplied diagrams. Each figure is referenced by name in the main documentation sections.

Figure 1. Use case model



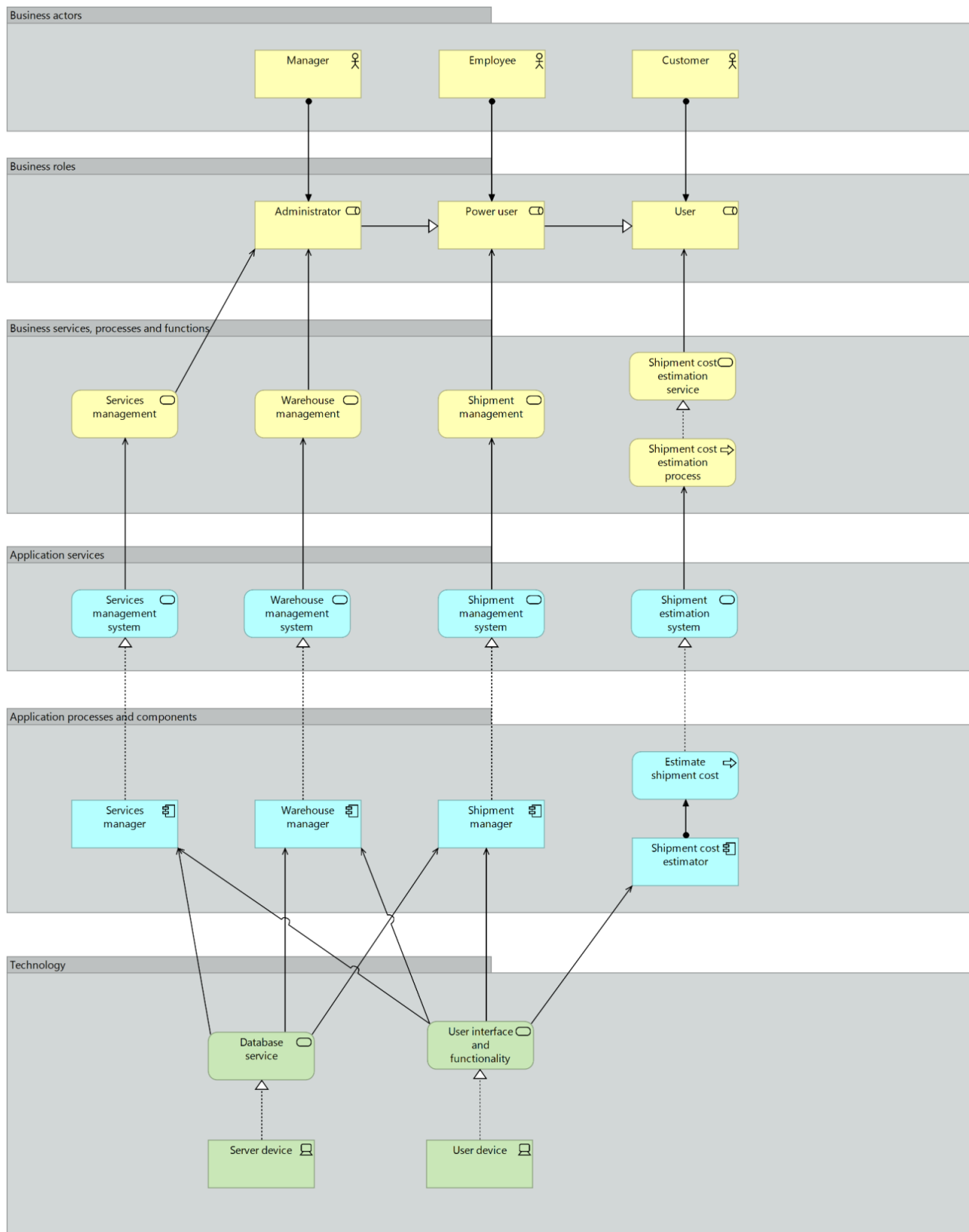
Shows the main actors and use cases.

Figure 2. Application usage viewpoint



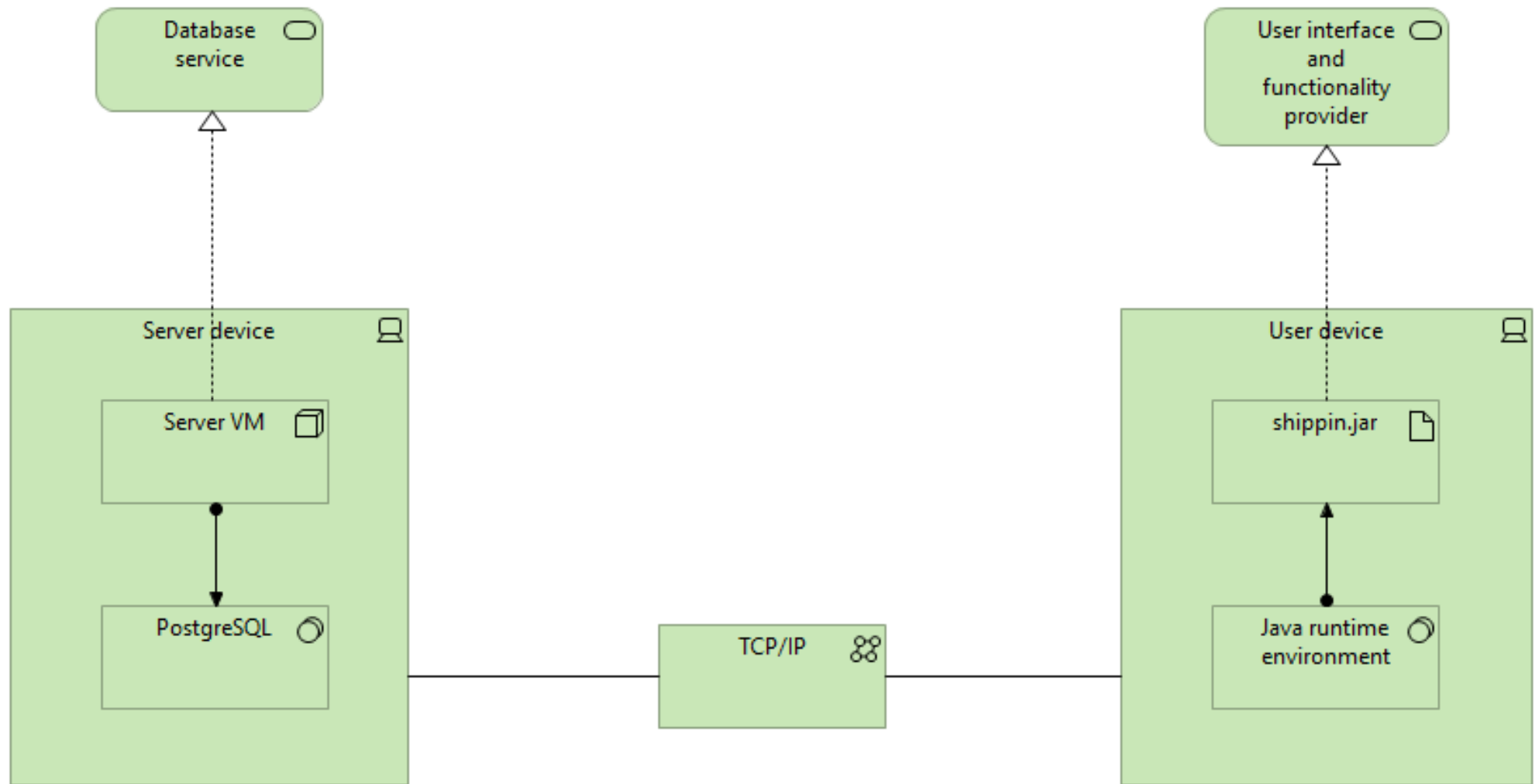
Connects business actors and roles with application services such as services management, warehouse management, shipment management and shipment estimation.

Figure 3. Layered viewpoint



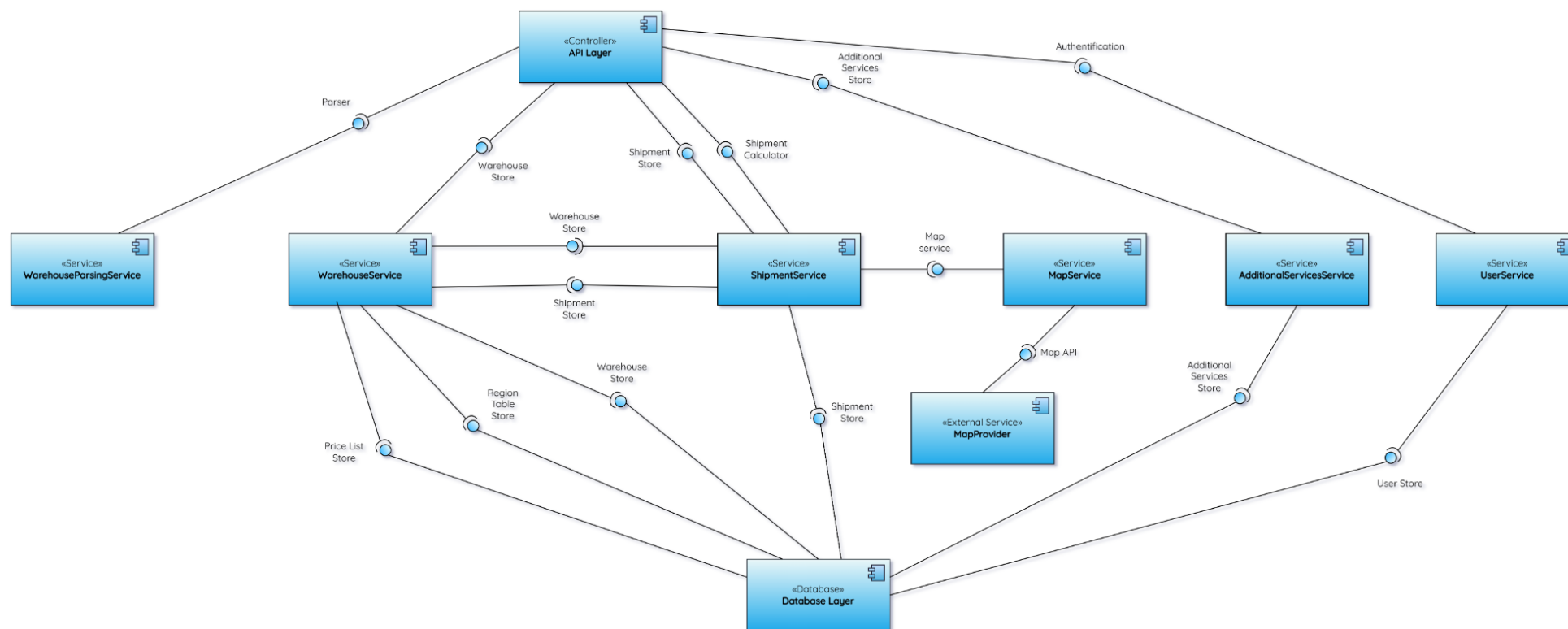
Shows how business roles, business services, application services, components and technology elements relate to each other.

Figure 4. Technology layer viewpoint



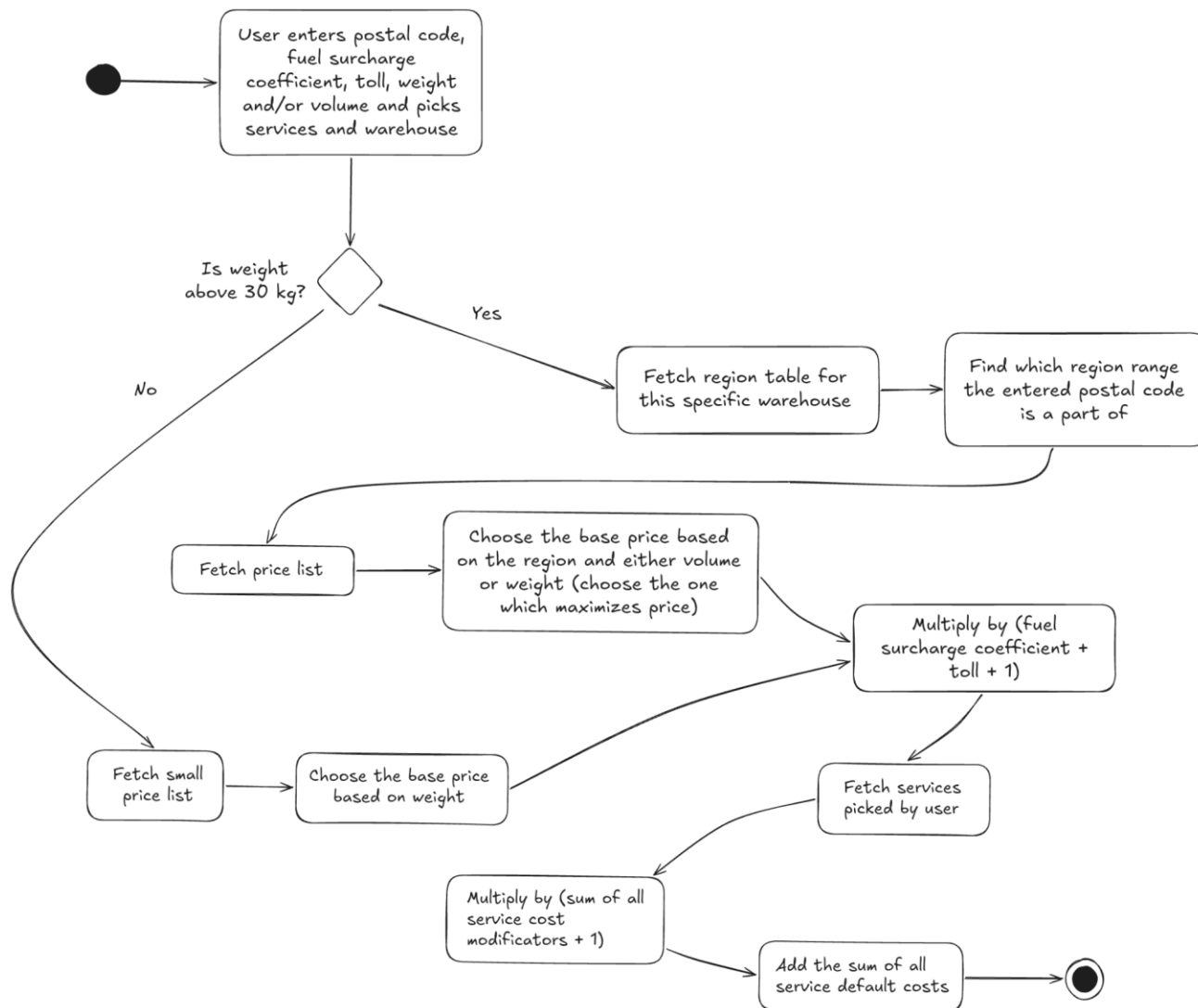
Shows the deployment idea: user device with Java runtime and JAR file communicating over TCP/IP with a server VM running PostgreSQL.

Figure 5. Component diagram - application services



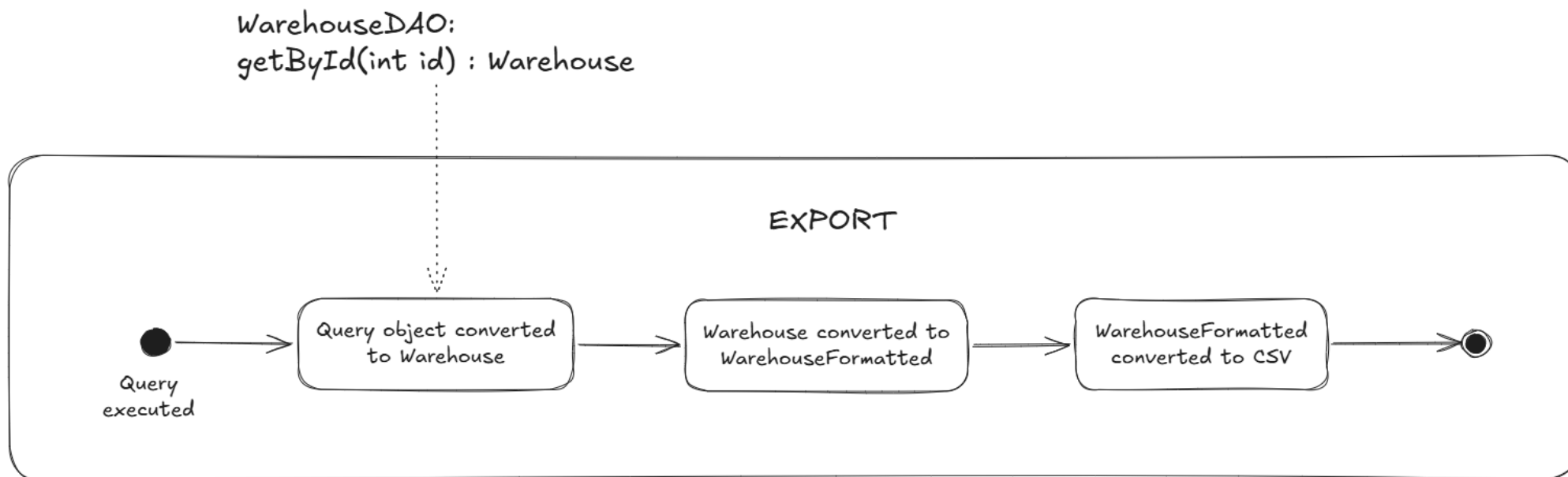
Shows the main service components, API/controller layer, external map provider and database layer.

Figure 6. Activity diagram - shipment cost estimation

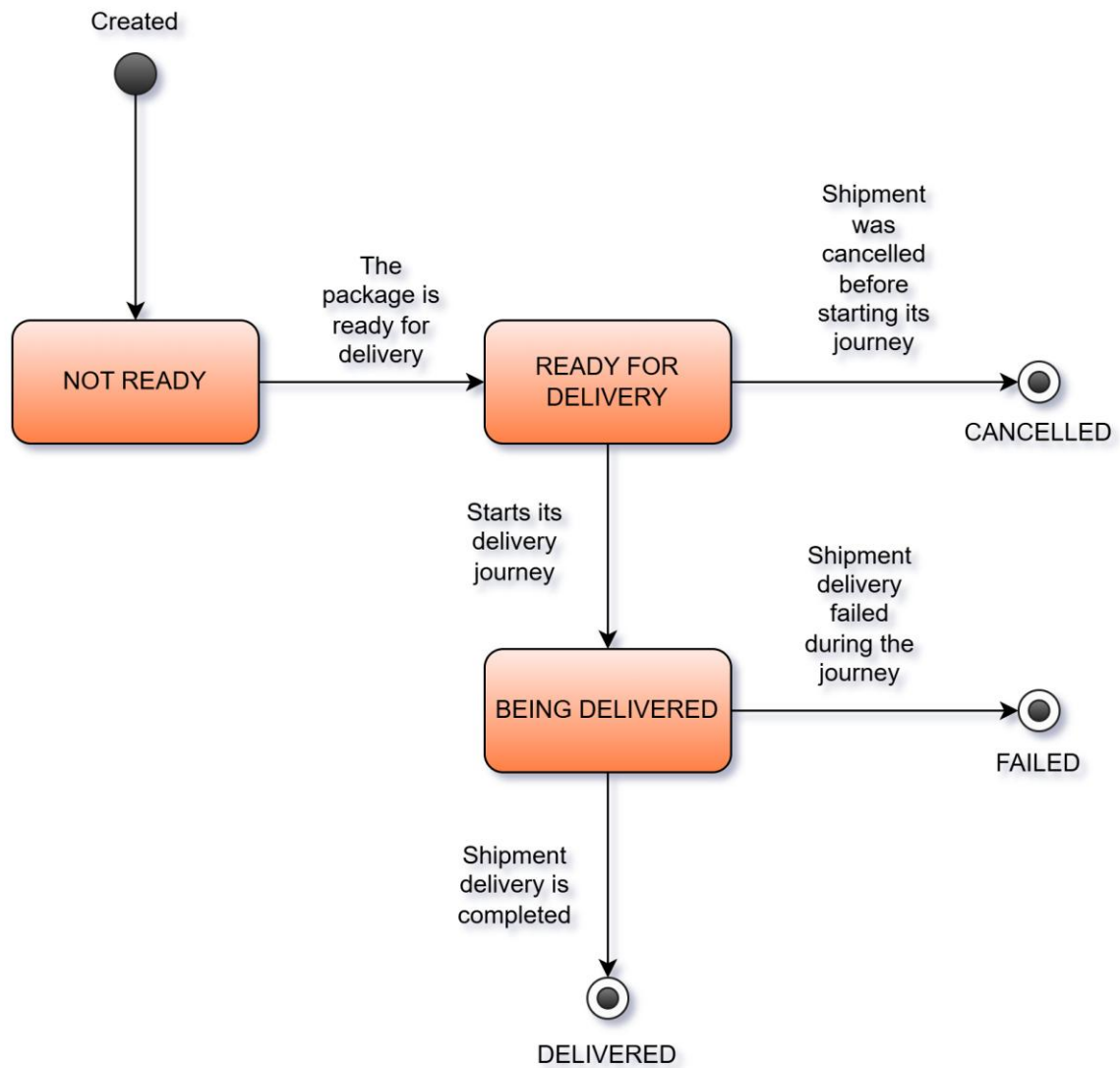


Shows the business steps of the shipment cost calculation process.

Figure 7. Activity diagram - warehouse export

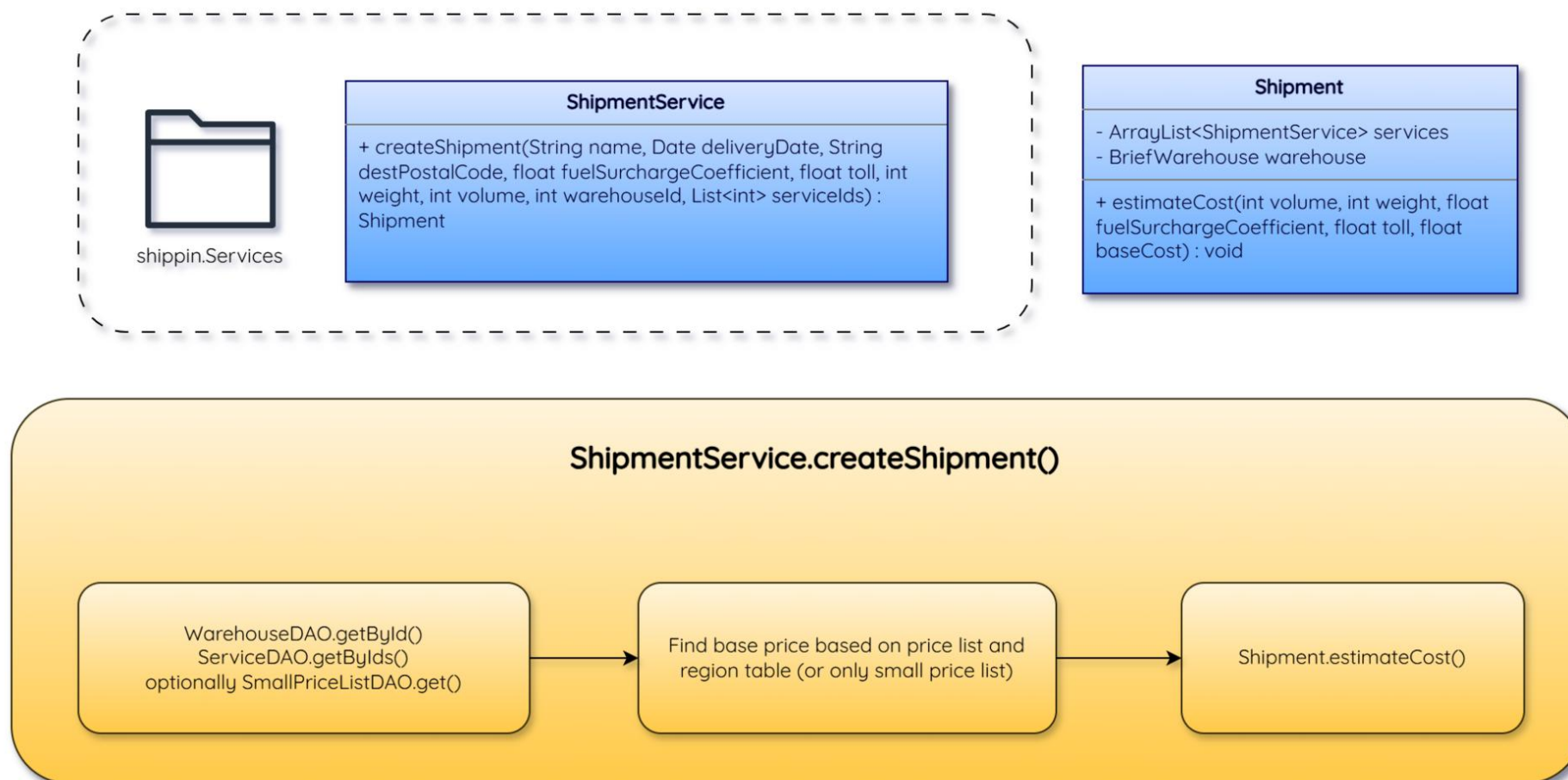


Shows the export process from query result through warehouse conversion and CSV creation.

Figure 8. State diagram - shipment lifecycle

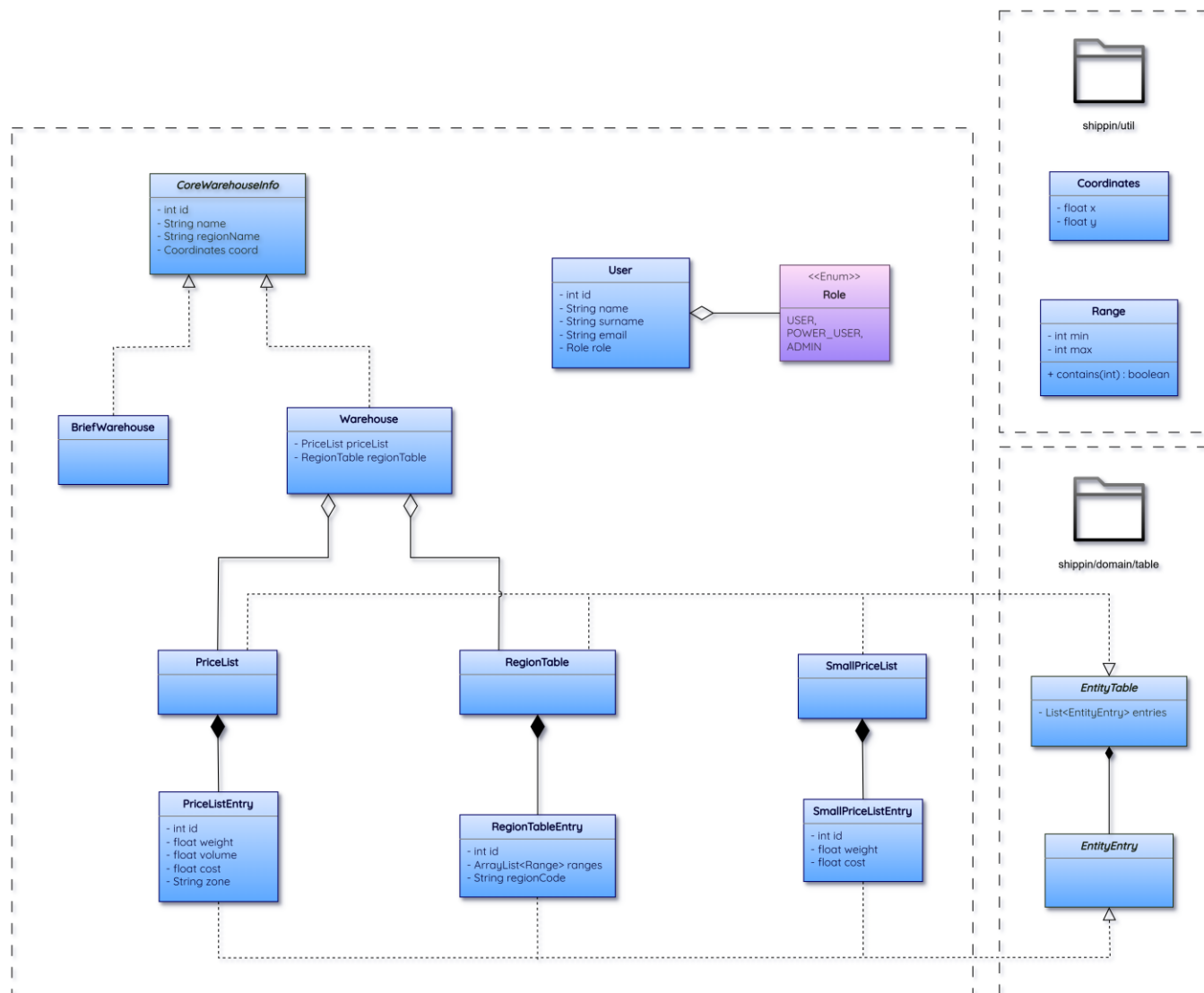
Shows permitted shipment states and transitions from creation to delivered/cancelled/failed outcomes.

Figure 9. Class diagram – ShipmentService cost estimation



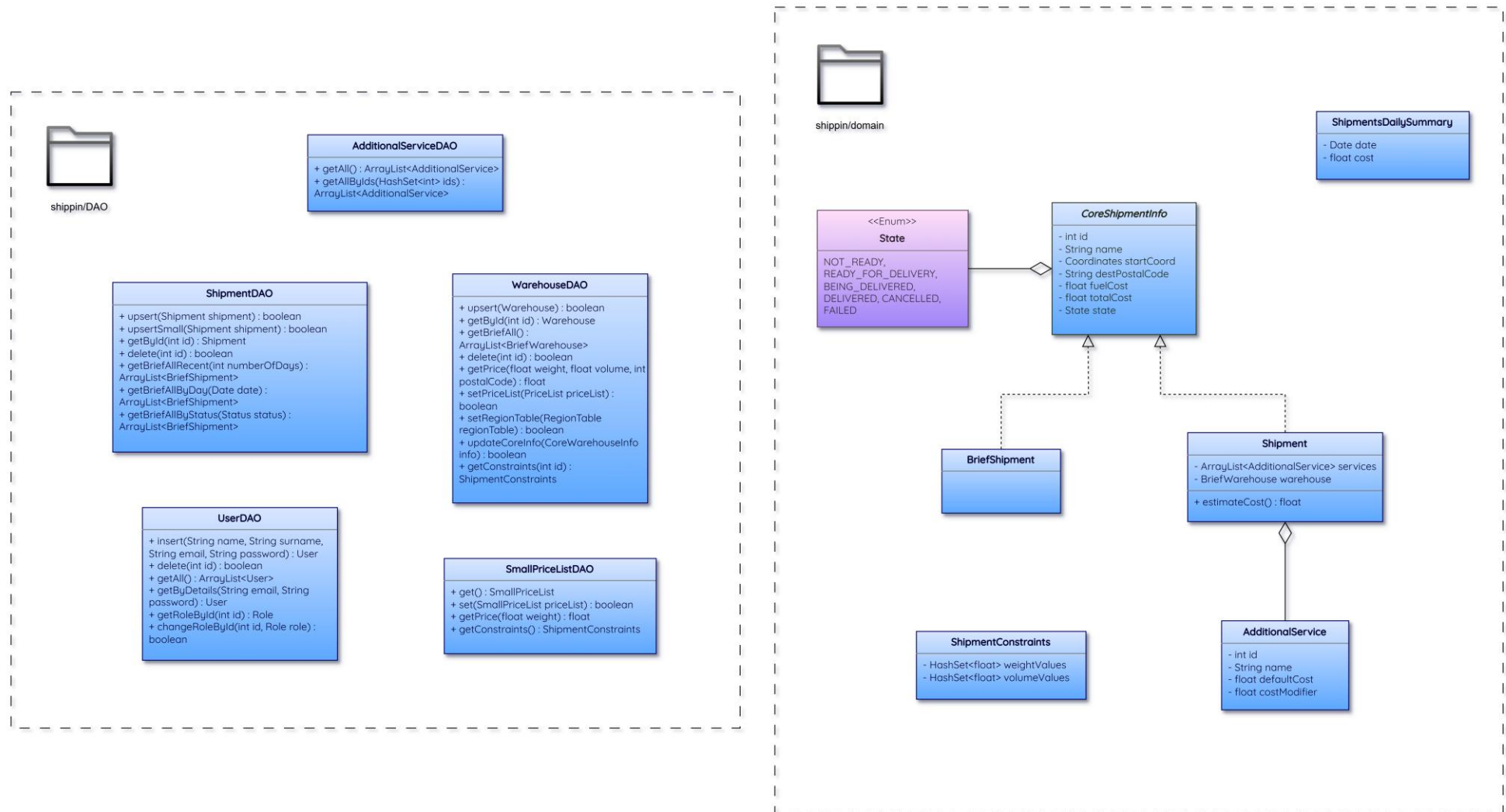
Shows the central role of ShipmentService.createShipment and the relation to Shipment and estimateCost behavior.

Figure 10. Class diagram – Warehouse and user domain objects



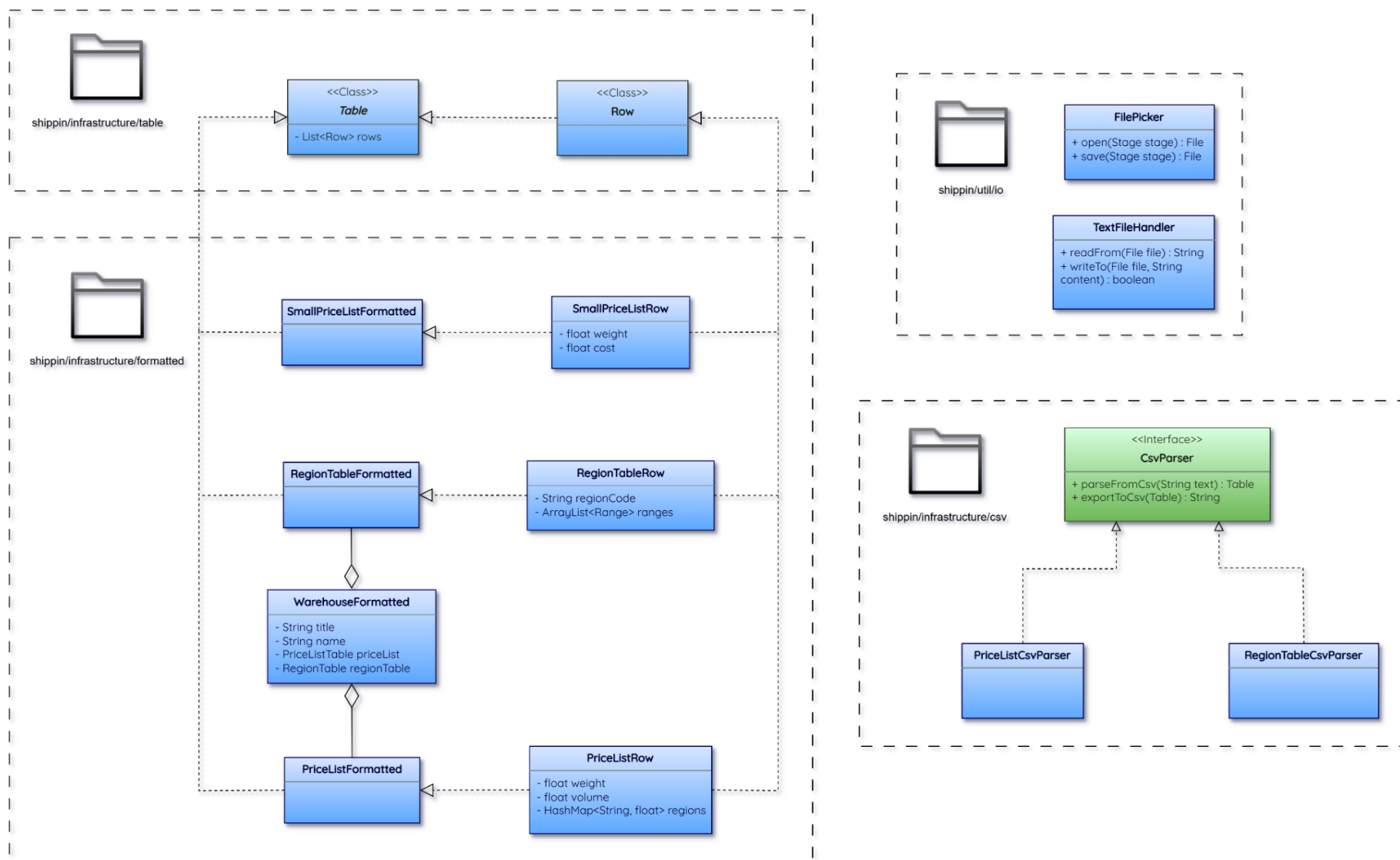
Shows the warehouse and user domain objects, as well as table abstractions and helper objects.

Figure 11. Class diagram – Shipment and ShipmentDailySummary domain objects, and DAO objects.



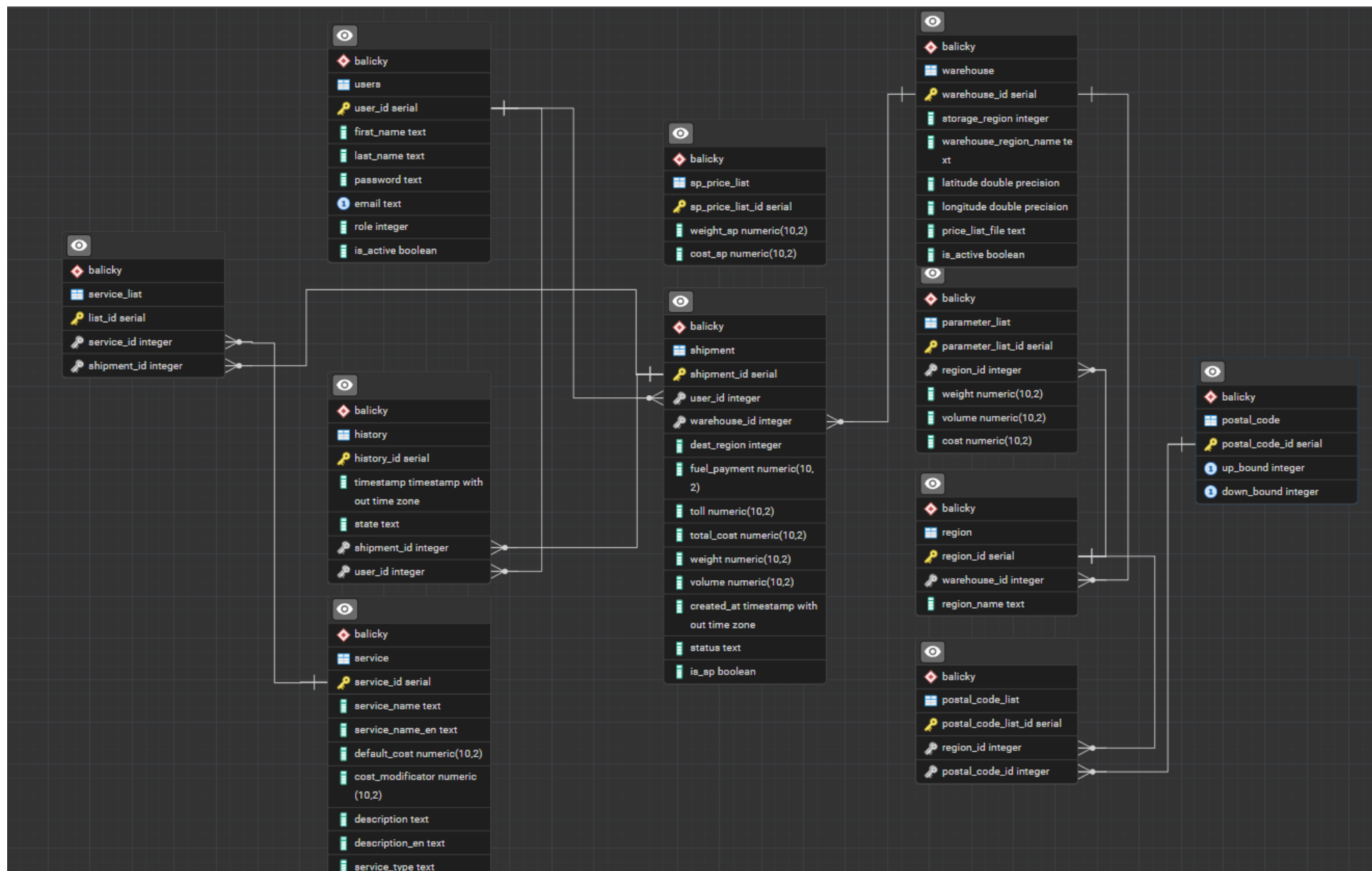
Shows DAO objects, shipment domain object and a ShipmentsDailySummary domain object.

Figure 12. Class diagram - CSV/table infrastructure



Shows formatted table classes, row classes, file utilities and CSV parser abstractions.

Figure 13. Database schema



Shows PostgreSQL tables for users, warehouses, shipments, history, services, price lists, regions and postal-code ranges.